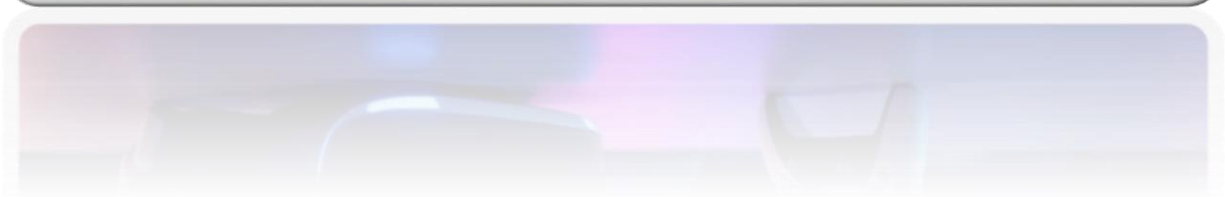




Challenge développement d'application BTS SIO2 SLAM 2025
Groupe 1 : La prairie pluvieuse

Kamdine, Sael, Quentin



Contexte :	3
Composition de l'équipe :	4
1er outil : Trello.....	5
2ème outil : Github.....	6
3ème outil : PlanetHoster.....	7
Contraintes :	8
Dossier de conception :	9
Diagramme de classes :	9
Diagramme de cas d'utilisation (use-case) :	10
Description textuelle du cas d'utilisation :	11
Scénario nominal :	11
Scénarios alternatifs :	11
Diagramme de Séquences de l'activité : « Enregistrer l'avatar »	12
Schéma d'architecture :	13
Documentation BDD :	14
Back-Office : Côté USER.....	15
Page du formulaire :	16
Le carousel « infini » des mondes :	17
Le formulaire de connexion :	18
Back-Office : Côté ADMIN.....	20
Environnement VR :	23
Présentation :	24
Ajout des fonctionnalités VR :	25
Exemple : Le système de pluie.....	25
Exemple : Le système d'interaction (grab).....	26
Pour la gestion et ajouts d'objets :	27
Majeur problème : La physique	30
Communication entre Back-Office et VR :	31
Problèmes rencontrés :	33
Conclusion	34
Conclusion du projet :	34
Conclusion générale du challenge :	34

Contexte :

CHOPINSOFT a prévu de lancer un nouveau concept d'environnement immersif en réalité virtuelle.

Ce concept permettrait aux utilisateurs de choisir dans un premier temps leur avatar puis de s'immerger dans un des cinq environnements immersifs intégré.

Les délais de production étant relativement courts, l'éditeur a décidé de faire participer 5 filiales différentes pour développer chacune un environnement immersif, propre à chaque thème.

Il était également demandé de développer **un back-office** pour permettre à l'utilisateur de créer son compte via une interface de type jeu-vidéo sur pc en suivant une maquette précise fournis.

Groupe 1 : La prairie pluvieuse

Groupe 2 : Le désert aride

Groupe 3 : La Laponie enneigée

Groupe 4 : Le brouillard de la forêt

Groupe 5 : La montagne escarpée

Nous sommes le groupe 1, ayant été désigné pour développer la **prairie pluvieuse**, un monde où l'utilisateur serait immergé dans une prairie calme, avec une maison, un enclos avec des animaux et des arbres autour pour accentuer l'immersion.

Prévisualisation fournie par notre professeur :



Composition de l'équipe :

3 membres :

- Sael SELLES => Chef de projet, qui s'est chargé de l'attribution des tâches, des merges sur Git, et des nombreux tests de la réalité virtuelle en plus du développement de fonctionnalités.
- Quentin JEANDEL => Développeur, conception des différents diagrammes pour initialiser le projet, grande contribution à la conception VR du projet et amélioration du design.
- Kamdine HASSANZADEH => Développeur, développement du back-office, rédaction des différents comptes rendu quotidien et hebdomadaire, ajout de différentes fonctionnalités et objets dans la VR.

Malgré la distribution des tâches respectives à chacun, ces mêmes tâches ont pu varier en cours du challenge **en fonction des compétences et difficultés de chacun.**

La répartition des tâches s'est faite via **un tableau Excel** afin de déterminer le responsable de chaque action et de qui l'effectue :

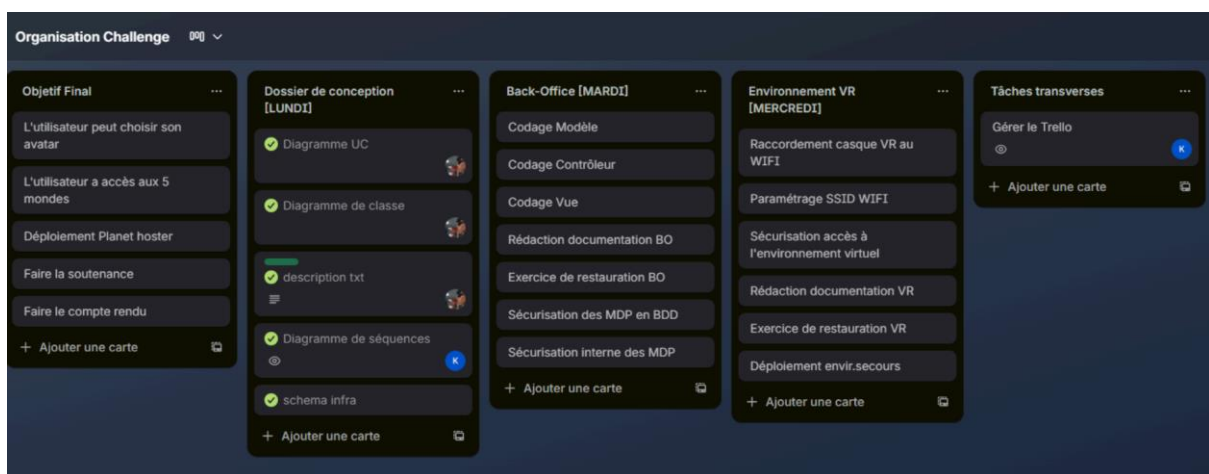
[illegible]

1er outil : Trello

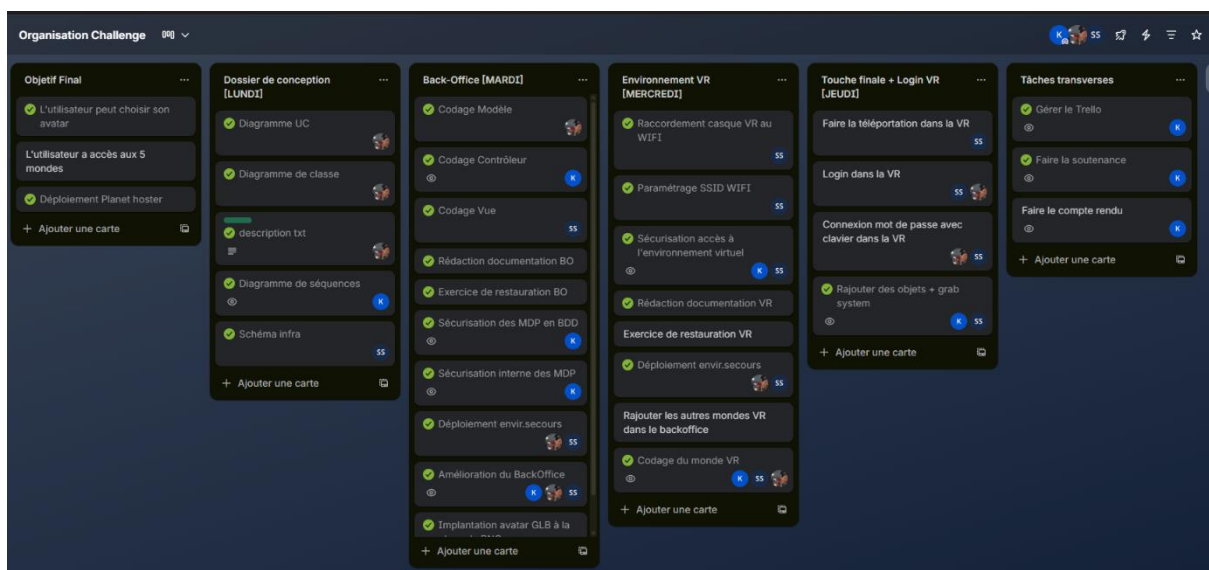


Cet outil de gestion de projet a permis de structurer notre organisation agile en répartissant les tâches, en fixant les échéances et en suivant l'avancement global de l'équipe via un tableau Kanban :

Trello en début de challenge :



Trello en fin de challenge :

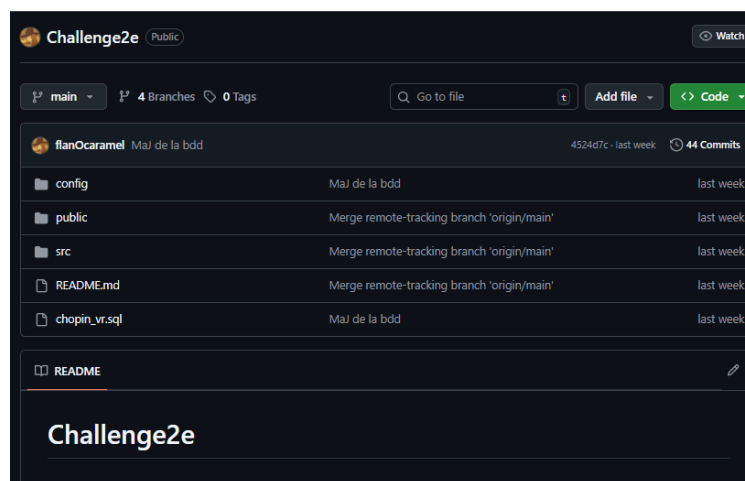


2ème outil : Github

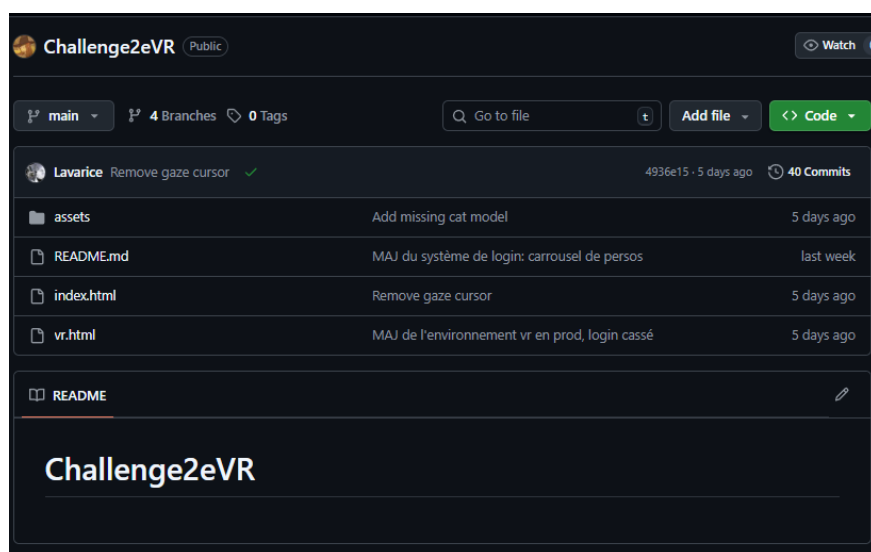


Cette plateforme a assuré le versionnage et la centralisation de notre code source, facilitant ainsi la collaboration simultanée des différents membres du groupe et la sécurisation de l'historique des modifications afin de revenir à des versions ultérieures en cas de soucis majeur (qui a souvent été le cas dans la VR)

Github du back-office :



Github de la VR :



3ème outil : PlanetHoster



Cette solution d'hébergement web a servi d'infrastructure de production pour déployer notre back-office, notre VR et notre base de données MySQL, rendant le service accessible en ligne.

PlanetHoster permet également de sécuriser et sauvegarder la base de données de notre application.

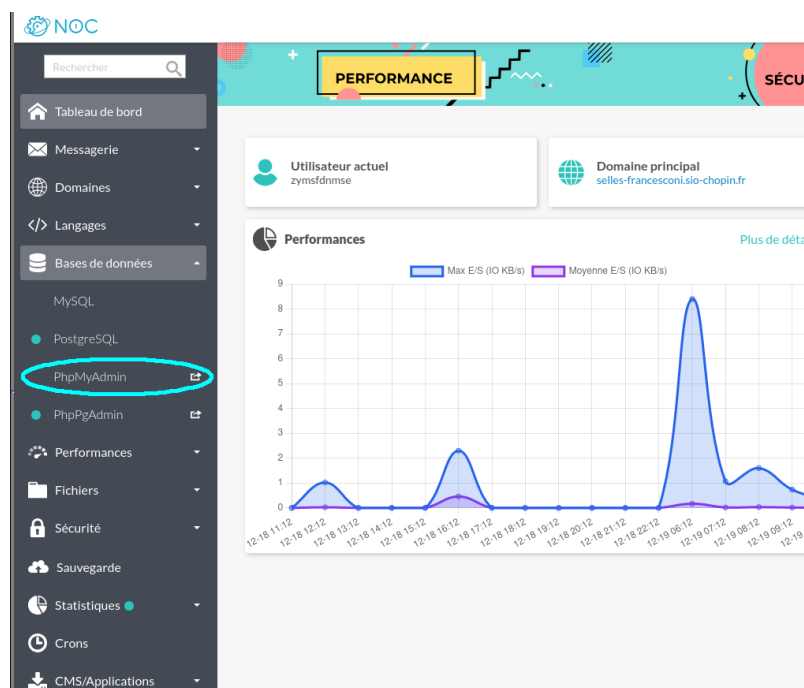
Lien de notre back-office (création de compte) :

<https://selles-francesconi.sio-chopin.fr/Challenge2e/public/>

Lien de notre VR (connexion et immersion dans le monde VR) :

<https://selles-francesconi.sio-chopin.fr/Challenge2eVR/>

Capture d'écran de l'interface NOC :



Contraintes :

Pour mener à bien ce projet, nous avons eu un cahier des charges présentant les différents critères de réussite :

Architecture de l'application :

- Back-office développé en PHP
- Arborescence qui respecte l'architecture MVC (Model-View-Controller)
- Base de données par MySQL
- L'application hébergé sur PlanetHoster fourni par le lycée.

Wifi et accès à internet :

- Le casque de réalité virtuelle devra utiliser la sortie vers internet Adista disponible sur le réseau du laboratoire SISR, pour accéder à Internet
- Paramétrer un SSID distinct sur le routeur Netgear mis à disposition pour le challenge

Sécurité / Sauvegarde :

- Solution de sauvegarde des applications et des données en ligne, via Github et PlanetHoster pour la BDD
- Les portails de réalités virtuelles devront être accessibles uniquement aux utilisateurs ayant créé un compte sur le back-office de l'application.
- Les mots de passe des accès doivent être hashés en base de données
- Dashboard Admin pour gérer les mondes et différents avatars du site, avec interface cachée et comptes déclarés sur MySQL.

Nous avons été disposés d'ordinateurs pour chaque étudiant équipé des différents logiciels de programmation et d'un **casque VR Meta Oculus Quest 1** pour tester en temps réel nos modifications sur le monde virtuel créée.

Dossier de conception :

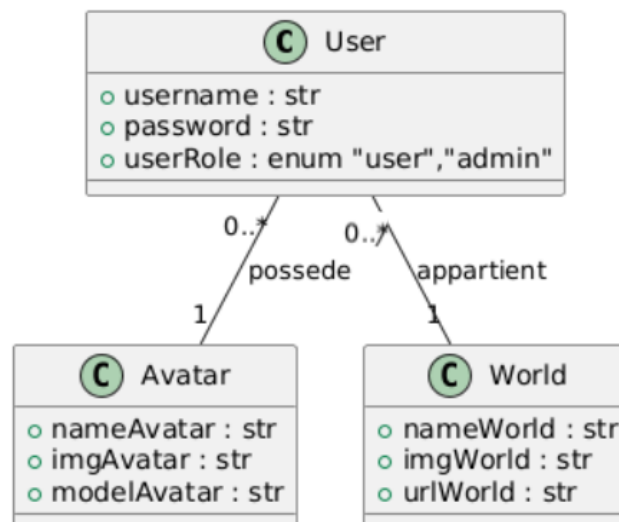
Pour garantir la réussite du développement dans les délais impartis, nous avons débuté le challenge par la réalisation d'un dossier de conception complet.

Cette étape a servi de feuille de route à toute l'équipe, permettant de valider l'architecture technique avant d'écrire la moindre ligne de code.

Diagramme de classes :

Ce diagramme a permis la cohérence des données lors de la création de la BDD.

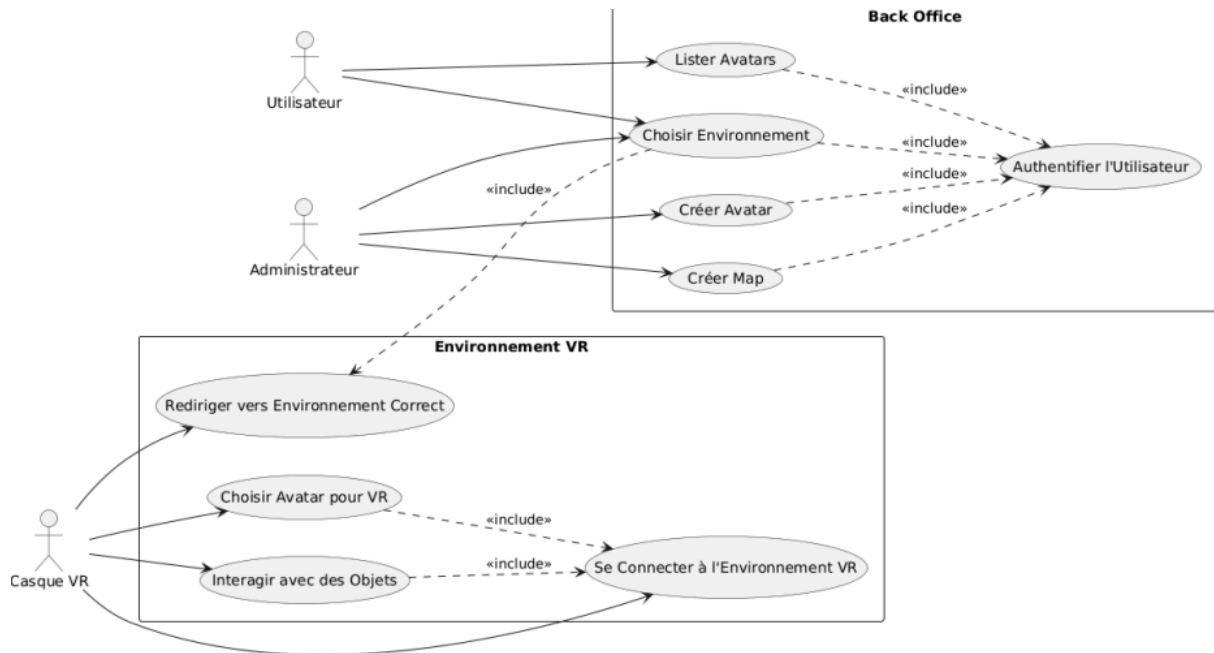
Bien que les différentes filiales soient en compétitions, nous avons eu un accord général avec l'ensemble des équipes pour partir sur la même base de données initiale afin, dans la finalité, de faciliter grandement la synchronisation des différents mondes et utilisateurs lors du déploiement.



Nous avons une classe User pour les utilisateurs qui ont un rôle entre user et admin, une pour les avatars qu'ils choisissent et une pour les différents mondes avec leur URL d'accès.

- Un utilisateur appartient à un seul mais un monde peut avoir plusieurs utilisateurs.
- Un utilisateur possède un avatar et un avatar peut posséder plusieurs utilisateurs

Diagramme de cas d'utilisation (use-case) :



Ce diagramme définit le périmètre fonctionnel du système en identifiant les acteurs (user et admin) et les actions qu'ils peuvent effectuer dans l'application

Ce diagramme structure deux environnements distincts, le back-office qui est dédié à l'administration (gérer les avatars et les mondes) et à la création de l'avatar (choix du monde, de l'avatar et de l'authentification).

Il est également précisé que l'authentification est obligatoire (« include ») pour sécuriser l'accès aux fonctionnalités de création de comptes.

Lorsque l'utilisateur sera identifié il pourra être redirigé vers l'environnement choisi dans la VR et interagir avec ce dernier.

Description textuelle du cas d'utilisation :

Cette description détaille étape par étape le déroulement d'un cas d'utilisation, en prévoyant le scénario idéal (nominal) ainsi que les erreurs possibles (alternatif).

Scénario nominal :

1. Description textuelle du cas d'utilisation : Scénario Nominal
2. L'utilisateur clique sur "Create Avatar".
3. L'utilisateur choisit un modèle 3D pour son avatar.
4. L'utilisateur choisit le monde dans lequel il souhaite aller.
5. L'utilisateur entre un nom d'utilisateur et un mot de passe.
6. Le mot de passe correspond aux critères (requirements) : l'avatar est créé.
7. L'utilisateur se dirige vers le menu Avatar VR.
8. L'utilisateur fait défiler les options avec les flèches G/D pour trouver son avatar.
9. L'utilisateur clique sur "Sélectionner l'avatar".
10. L'utilisateur vient du Monde 1.
11. Le clavier apparaît (*spawn*).
12. L'utilisateur écrit son mot de passe avec le clavier.
13. Quand l'utilisateur clique sur "Entrer", il est redirigé vers le monde.

Scénarios alternatifs :

Scénario alternatif : Le mot de passe ne respecte pas les critères (lors de la création)

- (4) L'utilisateur entre un nom d'utilisateur et un mot de passe.
- (5) Le mot de passe ne correspond pas aux critères. Il doit donc en entrer un autre qui respecte les exigences.

Scénario alternatif : L'utilisateur vient d'un autre monde

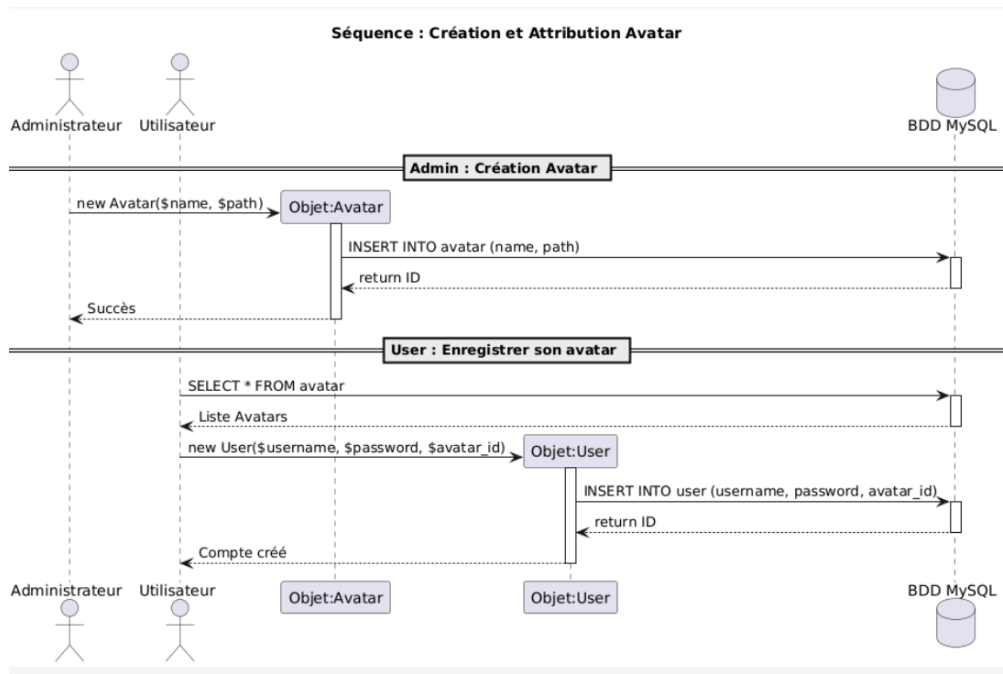
- (8) L'utilisateur clique sur "Sélectionner l'avatar".
- (9) L'utilisateur ne vient pas du Monde 1. Il est alors redirigé vers le bon monde.

Scénario alternatif : Le mot de passe est incorrect (lors de la connexion)

- (11) L'utilisateur écrit son mot de passe avec le clavier.
- (12) Le mot de passe ne correspond pas à l'avatar. Il doit donc resélectionner son avatar et entrer un mot de passe valide.

Diagramme de Séquences de l'activité : « Enregistrer l'avatar »

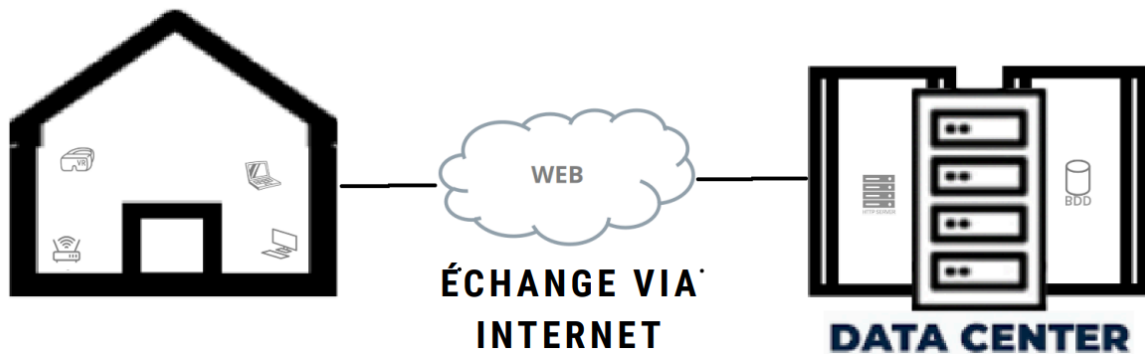
Ce diagramme illustre la chronologie des échanges techniques entre les acteurs, le système et la base de données pour réaliser une fonctionnalité précise.



Pour l'activité "Enregistrer Avatar", on montre le lien technique entre la préparation des données et leur utilisation. L'administrateur doit d'abord rendre les modèles d'avatars disponibles en base de données (INSERT INTO avatar). Ensuite, lors de l'inscription, l'utilisateur sélectionne l'un de ces avatars, liant ainsi son nouveau compte utilisateur à l'identifiant unique de l'avatar choisi via une requête SQL.

Schéma d'architecture :

Ce schéma permet de visualiser l'infrastructure physique et logique de la solution, en distinguant les équipements clients des ressources serveurs.



L'architecture met en évidence la séparation entre l'environnement utilisateur (Maison), équipé d'un poste de travail pour le Back Office et d'un casque VR pour l'immersion, et le centre de données (Data Center). Ces deux zones communiquent via Internet, reliant le matériel local au serveur web Apache et à la base de données MySQL hébergés chez PlanetHoster.

Documentation BDD :

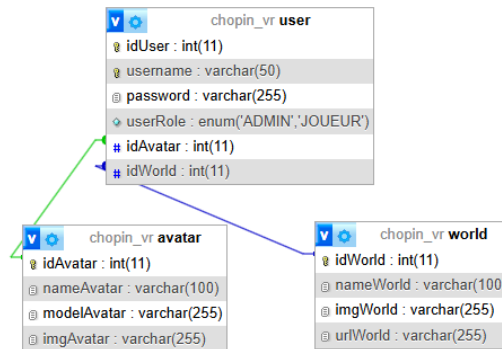


Table User

idUser INT (PK) : Id unique de l'utilisateur
username VARCHAR(50) : Nom de l'utilisateur
password VARCHAR(255): PWD hashé
userRole ENUM('user','admin') : rôle de l'utilisateur
idAvatar INT (FK) : référence à l'avatar
idWorld INT (FK) : référence au monde

Table Avatar

idAvatar INT (PK) : Id unique de l'avatar
nameAvatar VARCHAR(50) : Nom de l'avatar
imgAvatar VARCHAR(255) : URL qui redirige vers le PNG de l'avatar
modelAvatar VARCHAR(255) : URL qui redirige vers le GLB de l'avatar

Table World

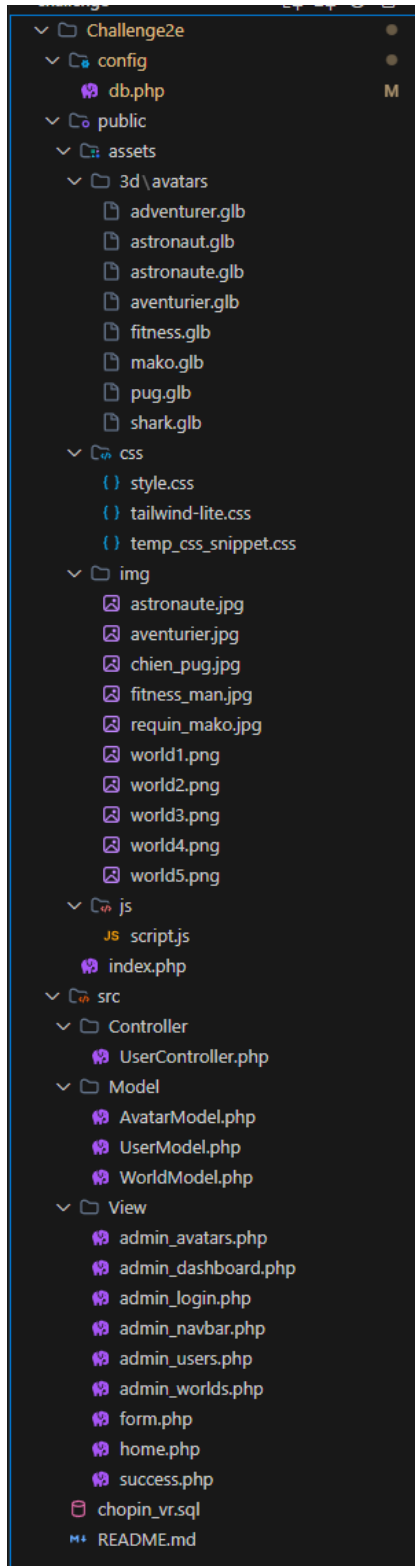
idWorld INT (PK) : identifiant unique de l'environnement
nameWorld VARCHAR(50) : nom de l'environnement
imgWorld VARCHAR(255) : URL qui redirige vers le PNG de l'environnement
urlWorld VARCHAR(255) : URL qui redirige vers le monde VR (5 différents)

Le dossier de conception est le plan d'architecte du projet, il nous permet de définir exactement quoi construire avant d'écrire la moindre ligne de code afin d'anticiper les erreurs, cette approche permet de réduire les coûts en qu'à d'erreur plus tard.

Enfin, il garantit que toute l'équipe ait la même vision et travaille sur des bases communes pour que les différentes parties du projet s'emboîtent parfaitement

Back-Office : Côté USER

Le projet repose sur une architecture MVC (Modèle-Vue-Contrôleur) structurée pour séparer les responsabilités et faciliter la maintenance. Voici le rôle de chaque dossier clé :



Le dossier config contient la configuration de l'application soit la connexion à la BDD via PDO

Le dossier public est le point d'entrée unique de l'application, c'est le dossier accessible depuis le navigateur, il contient l'index.php qui reçoit toutes les requêtes ainsi que les ressources statiques (assets/) et le style/script/glb de l'application

Le dossier src est le cœur applicatif, c'est ici que réside toute la logique métier, divisée selon le pattern MVC :

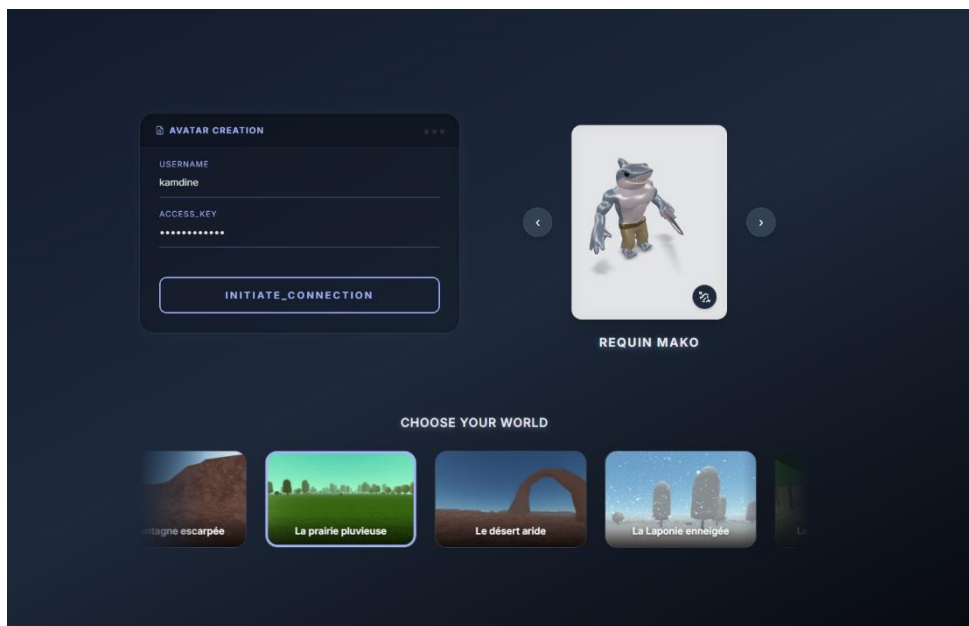
Le contrôleur reçoit les requêtes de l'utilisateur, demande les données aux modèles et sélectionne la vue appropriée à l'affichage (par exemple UserController.php gère la création de compte et le login admin via routage)

Le modèle gère les interactions avec la base de données. Chaque table a sa classe correspondante (par exemple AvatarModel.php contient les méthodes php pour modifier les avatars grâce au findAll)

La vue est l'ensemble des fichiers templates (HTML/PHP) qui présentent l'interface à l'utilisateur. Ils ne contiennent aucune logique métier complexe seulement de l'affichage (par exemple form.php est notre page de création d'avatar)

Les fichiers GLB sont les animations 3D des avatars pour donner vie au site et donner un aperçu à l'utilisateur de l'avatar qu'il s'apprête à choisir

Page du formulaire :



Pour la gestion des animations 3D :

L'intégration 3D repose sur le composant web standard `<model-viewer>`, qui permet d'afficher et d'interagir avec des fichiers `.glb` directement dans le navigateur. Un fichier `.glb` peut contenir plusieurs pistes d'animation (exemple: Idle, Run, Walk) encapsulées dans le même fichier binaire. Le composant `<model-viewer>` expose une API JavaScript permettant d'accéder à ces animations via la propriété `availableAnimations`.

```
/* ===== ANIMATION TOGGLE ===== */
animBtn.onclick = () => {
  const availableAnims = avatarModel.availableAnimations;
  if (availableAnims && availableAnims.length > 0) {
    // Get current animation name or default to none
    const currentAnim = avatarModel.animationName;

    let nextIndex = 0;
    if (currentAnim) {
      const currentIndex = availableAnims.indexOf(currentAnim);
      if (currentIndex !== -1) {
        nextIndex = (currentIndex + 1) % availableAnims.length;
      }
    }

    avatarModel.animationName = availableAnims[nextIndex];
    console.log("Switched animation to:", availableAnims[nextIndex]);
  } else {
    console.log("No animations available for this model.");
  }
};

document.getElementById("prevBtn").onclick = () => {
  currentAvatar = (currentAvatar - 1 + avatars.length) % avatars.length;
  renderAvatar();
};

document.getElementById("nextBtn").onclick = () => {
  currentAvatar = (currentAvatar + 1) % avatars.length;
  renderAvatar();
};

renderAvatar();
```

Le carousel « infini » des mondes :

Pour la sélection des mondes, nous avons mis en place un carousel à défilement infini qui permet à l'utilisateur de faire la navette entre les différentes options (La Prairie, Le Désert etc...

L'illusion de l'infini est créée par une technique classique qui consiste à dupliquer le contenu, le conteneur « carousel-track » contient deux fois la liste des mondes, une animation css déplace le contenu vers la gauche et dès que le premier groupe a fini de défiler l'animation se réinitialise instantanément à 0, créant une boucle infinie à œil nu

```
/* ===== DRAG TO SLIDE ===== */
const carouselContainer = document.querySelector('.infinite-carousel');
const carouselGroups = document.querySelectorAll('.carousel-group');

let isDown = false;
let startX;
let lastX;

// Helper to control animation
function pauseAnimations() {
  carouselGroups.forEach(group => {
    group.getAnimations().forEach(anim => anim.pause());
  });
}

function playAnimations() {
  carouselGroups.forEach(group => {
    group.getAnimations().forEach(anim => anim.play());
  });
}

function scrubAnimations(deltaX) {
  carouselGroups.forEach(group => {
    group.getAnimations().forEach(anim => {
      // Sensitivity: 15ms per pixel
      let newTime = anim.currentTime - (deltaX * 15);

      // NORMALIZE TIME to [0, 20000]
      const duration = 20000;
      newTime = (newTime % duration + duration) % duration;

      anim.currentTime = newTime;
    });
  });
}

// Hover Effects
carouselContainer.addEventListener('mouseenter', () => {
  if (!isDown) pauseAnimations();
});
carouselContainer.addEventListener('mouseleave', () => {
  if (!isDown) playAnimations();
});

// Drag Interaction
carouselContainer.addEventListener('mousedown', (e) => {
  isDown = true;
  startX = e.pageX;
  lastX = startX;
  carouselContainer.style.cursor = 'grabbing';
  pauseAnimations();
});
```

On a également la possibilité de « drag » le carousel pour le rendre plus rapide, l'utilisateur peut déplacer manuellement le carousel grâce à la manipulation du temps de l'animation, rendant le site plus interactif

Le formulaire de connexion :

Au lieu d'un formulaire classique, nous avons opté pour une approche imitant un terminal de commande futuriste type Cyberpunk avec un vocabulaire thématique comme ACCESS_KEY pour donner la sensation qu'on accède vraiment à un monde différent, puis l'effet glitch pour renforcer l'immersion.

Voici la structure classique qui permet cet affichage :

```
<div class="glitch-card">
  <div class="card-header">
    <div class="card-title">AVATAR CREATION</div>
    <!-- Éléments décoratifs pour le touche futuriste -->
    <div class="card-dots"><span></span><span></span><span></span><span></span></div>
  </div>

  <div class="card-body">
    <!-- Champ Username avec label animé -->
    <div class="form-group">
      <input type="text" id="username" name="username" required placeholder=" " />
      <label for="username" class="form-label" data-
text="USERNAME">USERNAME</label>
    </div>

    <!-- Champ Password renommé en "ACCESS_KEY" pour l'immersion -->
    <div class="form-group">
      <input type="password" id="password" name="password" required placeholder="
" />
      <label for="password" class="form-label" data-
text="ACCESS_KEY">ACCESS_KEY</label>
    </div>

    <!-- Bouton Cyberpunk -->
    <button data-text="INITIATE_CONNECTION" type="submit" class="submit-btn">
      <span class="btn-text">INITIATE_CONNECTION</span>
    </button>
  </div>
</div>
```

Nous avons également ajouté des conditions de sécurité lors de l'inscription quand l'username est déjà utilisé, que le mot de passe n'est pas assez sécurisé etc... :

```

// Traitement du formulaire de création
public function create()
{
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        // Vérification CSRF
        if (!isset($_POST['csrf_token']) || $_POST['csrf_token'] !== $_SESSION['csrf_token']) {
            die('CSRF attack detected');
        }

        // Récupération des champs
        $username = $_POST['username'] ?? '';
        $password = $_POST['password'] ?? '';
        $idAvatar = $_POST['idAvatar'] ?? null;
        $idWorld = $_POST['idWorld'] ?? null;

        // Validation basique
        if (!empty($username) && !empty($password) && !empty($idAvatar) && !empty($idWorld)) {

            // Validation du mot de passe (8 chars, 1 maj, 1 chiffre, 1 spécial)
            if (!preg_match('/^(?=.*[A-Z])(?=.*\d)(?=.*[\W_]).{8,}$/', $password)) {
                $error = "The password must contain at least 8 characters, one uppercase letter, one number, and one special character.";
                $this->createAvatarForm($error, $_POST); // Recharge avec erreur et données
                return; // Arrêt du script
            }

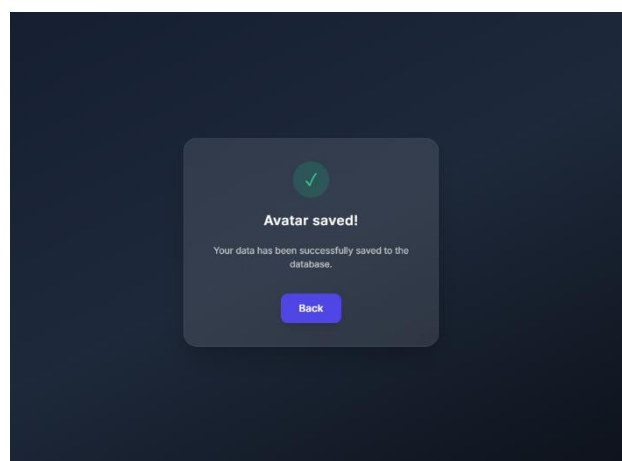
            // Hachage du mot de passe pour la sécurité (conformément aux contraintes)
            $hashed_password = password_hash($password, PASSWORD_ARGON2ID);

            // Vérification si le pseudo existe déjà
            if ($this->userModel->findByUsername($username)) {
                $error = "This username is already taken. Please choose another one.";
                $this->createAvatarForm($error, $_POST);
                return;
            }

            // Tentative de création
            if ($this->userModel->create($username, $hashed_password, $idAvatar, $idWorld)) {
                // Redirection vers la page de succès
                header("Location: index.php?page=success");
                exit();
            } else {
                $error = "Error, maybe you should try again.";
                // On recharge le formulaire avec l'erreur
                $this->createAvatarForm($error, $_POST);
            }
        } else {
            $error = "All fields are required.";
            $this->createAvatarForm($error, $_POST); // Recharge le formulaire si incomplet
        }
    }
}

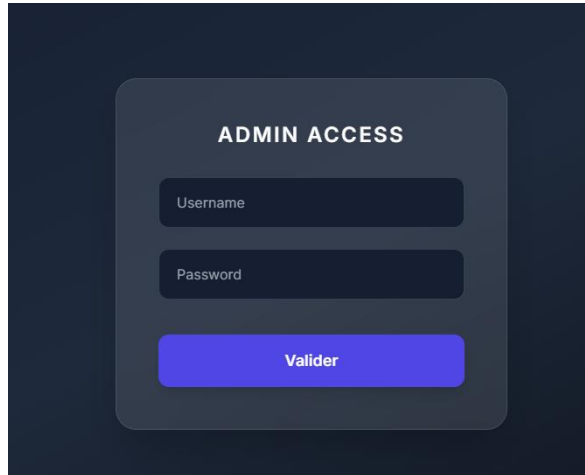
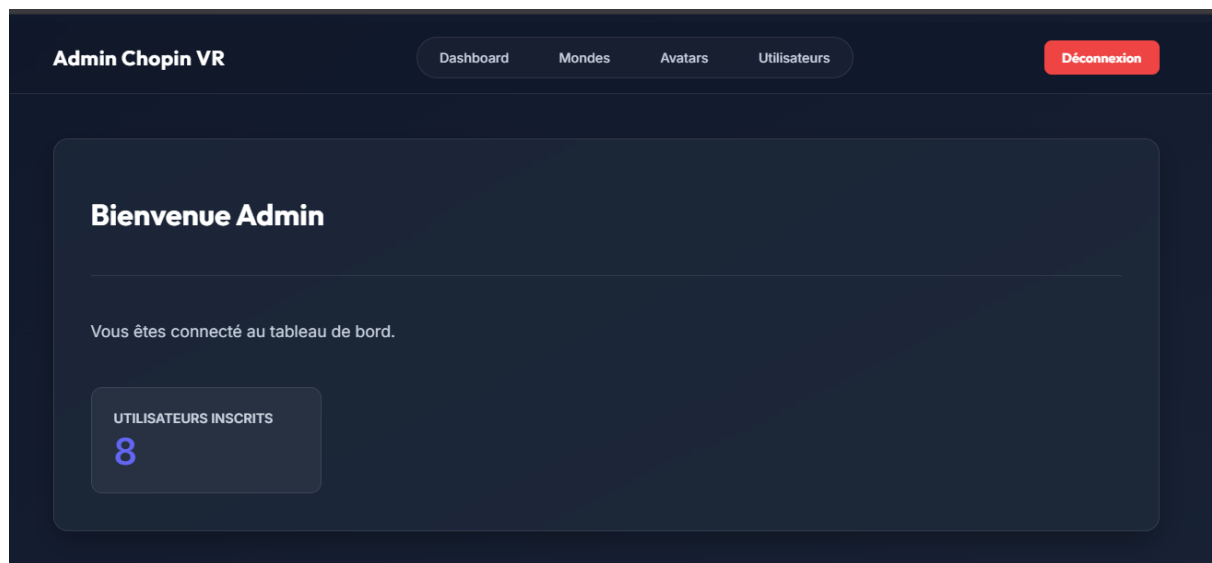
```

Quand l'utilisateur est inscrit, on le renvoie vers une page de confirmation et son compte est directement rajouté à la table user dans la BDD :



Back-Office : Côté ADMIN

Nous avons également une section strictement réservée **aux ADMINS** de l'application, avec un lien que seulement eux possèdent et des comptes créés directement à la BDD, la touche est invisible sur l'interface car quasi aucun jeu vidéo réel ne propose une touche de connexion administrateur lors de la connexion :

A dark-themed login form titled "ADMIN ACCESS". It features two input fields: "Username" and "Password", both with placeholder text. Below the fields is a blue button labeled "Valider". The form is centered on a dark background.

L'admin a ainsi la possibilité de voir le nombre d'utilisateurs inscrits, les différents mondes, les différents avatars et la liste des utilisateurs inscrits. Il peut également décider de supprimer ou ajouter des mondes ou les modifiant, via une interface interactive pour faciliter le processus :

Gestion des Mondes

Ajouter un monde

ID	NOM	IMAGE	URL	ACTIONS
1	La prairie pluvieuse		https://vr.chopin.com/world1	Modifier Supprimer
2	Le désert aride		https://vr.chopin.com/world2	Modifier Supprimer
3	La Laponie enneigée		https://vr.chopin.com/world3	Modifier Supprimer
4	Le brouillard de la forêt		https://vr.chopin.com/world4	Modifier Supprimer
5	La montagne escarpée		https://vr.chopin.com/world5	Modifier Supprimer

Exemple pour ajouter un monde on remplit les champs conformément à la BDD :

Gestion des Mondes

Ajouter un monde

Ajouter un nouveau monde

Nom du monde

test

Chemin de l'image

test

URL du monde

https://vr.chopin.com/world6

Ajouter le monde

Annuler

Qui sera ensuite rajouté à la BDD et défilera ainsi dans le carrousel de connexion, afin de garantir une implantation fluide de nouveaux contenus en cas de mise à jour et nouveautés :

Éditer

Copier

Supprimer

6 test

test

https://vr.chopin.com/world6

Liste des avatars avec les mêmes possibilités de gestion :

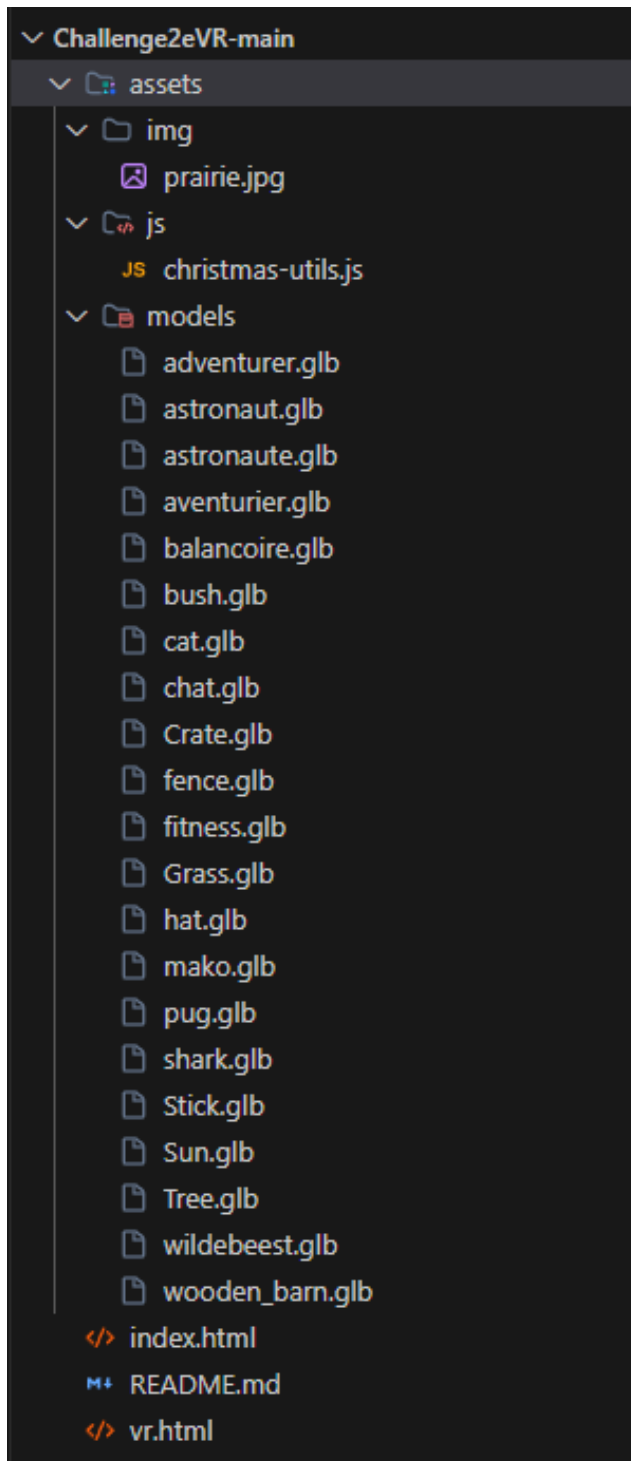
Gestion des Avatars					Ajouter un avatar
ID	NOM	MODÈLE 3D	IMAGE	ACTIONS	
1	Aventurier	adventurer.glb		Modifier	Supprimer
2	Astronaute	astronaut.glb		Modifier	Supprimer
3	Fitness	fitness.glb		Modifier	Supprimer
4	Requin Mako	shark.glb		Modifier	Supprimer
5	Chien Pug	pug.glb		Modifier	Supprimer

Liste des utilisateurs avec quelques informations :

Gestion des Utilisateurs					
ID	NOM D'UTILISATEUR	RÔLE	ID AVATAR	ID MONDE	ACTIONS
1	kamdinee	JOUEUR	1	1	Modifier Supprimer
2	quentin	JOUEUR	1	1	Modifier Supprimer
3	ryan	JOUEUR	1	1	Modifier Supprimer
4	kamdineeee	JOUEUR	1	1	Modifier Supprimer
5	quentinn	JOUEUR	1	1	Modifier Supprimer
8	ADMIN	ADMIN	1	1	Modifier
9	kamdineeeee	JOUEUR	4	1	Modifier Supprimer
10	aaaaaaaaaaaaa	JOUEUR	1	1	Modifier Supprimer

Environnement VR :

Le projet adopte une architecture Client-Side. Il s'agit d'une application VR web statique qui consomme une API externe pour les données dynamiques.



Le dossier assets contient :

- le dossier img avec les différentes images donc en l'occurrence l'affichage des mondes à la connexion

- le dossier js avec les scripts de logique métier spécifiques comme christmas-utils.js pour le comportement avancé des objets

- le dossier models stocke les actifs 3D au format .glb (avatars, décors, objectifs interactifs) afin d'enrichir le monde virtuel

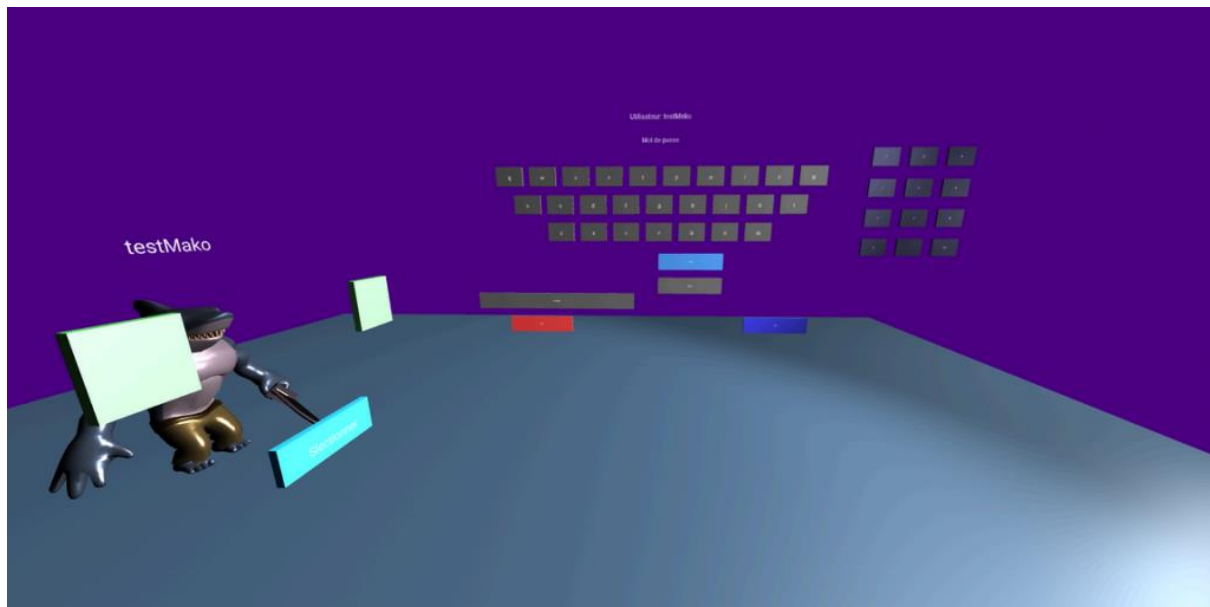
A la racine on a les points d'entrée de l'application :

- index.html est le hub d'accueil VR. Il gère l'authentification et la sélection des utilisateurs. Il contient une logique Js importante pour interagir avec l'API et gérer l'interface utilisateur (Carousel, Clavier Virtuel)

- vr.html est la scène principale de l'expérience « La Prairie Pluvieuse ». C'est ici qu'on définit la physique (qui n'a pas fonctionné en l'occurrence) et les éléments interactifs du monde

Présentation :

L'utilisateur branche sa VR, arrive dans l'hub de connexion index.html, les données de l'utilisateur sont récupérées via PHP depuis la base de donnée MySQL puis injectées dynamiquement dans la scène A-FRAME :



Il peut sélectionner son avatar via une sélection et doit ensuite se connecter à ce dernier pour accéder au monde à l'aide d'un clavier visuel avec un mot de passe crypté.

Ce après quoi il accédera au monde sélectionné (ici la prairie pluvieuse)



Ajout des fonctionnalités VR :

La partie VR utilise **le Framework A-Frame**, une surcouche à Three.js qui permet de créer des scènes 3D via du HTML déclaratif, couplé à un moteur physique (Ammo.js).

Comme ajout nous avons été chargés d'ajouter un système de pluie, d'enclos avec des gnous, une maison etc...

Exemple : Le système de pluie

L'effet de pluie est géré par un composant personnalisé rain-drop. Plutôt que d'utiliser un système de particules lourd, nous recyclons un nombre défini de gouttes pour optimiser les performances.

Logique : Chaque goutte tombe à chaque boucle de rendu (tick). Lorsqu'elle passe sous le sol ($y < 0$), elle est réinitialisée à une position aléatoire en haut (reset).

```
AFRAME.registerComponent("rain-drop", {
  init: function () {
    this.reset();
  },
  tick: function () {
    // La goutte tombe (On diminue Y)
    this.el.object3D.position.y -= 0.1;

    // Si elle touche le sol, on la recycle en haut
    if (this.el.object3D.position.y < 0) {
      this.reset();
    }
  },
  reset: function () {
    // Position aléatoire dans une zone de 20x20 autour du joueur
    var x = (Math.random() - 0.5) * 20;
    var z = (Math.random() - 0.5) * 20;
    var y = 10 + Math.random() * 5; // Hauteur entre 10 et 15m
    this.el.object3D.position.set(x, y, z);
  }
});
```

Les performances étant limitées la pluie est très minime (200)

```
<script>
  // Génération Pluie
  function createRain() {
    var scene = document.querySelector("a-scene");
    var rainCount = 200;
```

Exemple : Le système d'interaction (grab)

Pour attraper des objets, nous avons pour objectif à la base d'utiliser deux composants complémentaires : grabbable (sur l'objet) et grabber (sur la main/manette).

Fonctionnement :

1. Détection : Le grabber détecte une collision ou un clic sur un objet grabbable.
2. Attachement : L'objet devient "kinematic" (ne subit plus la gravité temporairement) et est "parenté" à la main pour suivre ses mouvements parfaitement.
3. Relâchement : L'objet reprend sa physique normale et retombe (non fonctionnel à la fin du challenge)

```
AFRAME.registerComponent("grabbable", {
  init: function () {
    this.grabbedBy = null;
  },
  grab: function (byEl) {
    if (this.grabbedBy) return false;
    this.grabbedBy = byEl;
    this.el.classList.add("is-grabbed");
    // Émettre un événement pour d'autres scripts (ex: sons)
    this.el.emit("grab-start", { by: byEl });
    return true;
  },
  release: function () {
    this.grabbedBy = null;
    this.el.classList.remove("is-grabbed");
    this.el.emit("grab-end");
  }
});
```

Cette fonctionnalité a été celle qui a pris le plus de temps à l'ensemble de l'équipe sans pouvoir fonctionner à l'issu du projet, par manque de compétence et majoritairement par manque de temps il était complexe de vérifier chaque modification dans la VR car la physique sur PC est impossible (tout est transparent dans l'aperçu navigateur)

Pour la gestion et ajouts d'objets :

L'intégration d'objets 3D se fait en deux étapes : le chargement des ressources et leur placement dans la scène.

1. Chargement (Assets) :

Nous préchargeons les modèles .glb dans la balise <a-assets> pour éviter les lags.

```
<a-assets>
  <a-asset-item id="wildebeestModel" src="./assets/models/wildebeest.glb"></a-asset-item>
  <a-asset-item id="barnModel" src="./assets/models/wooden_barn.glb"></a-asset-item>
</a-assets>
```

2. Placement dans la Scène :

Les objets sont ajoutés via des balises <a-entity>. Pour la végétation (arbres, herbe), nous utilisons un script procédural (createVegetation) qui génère des positions aléatoires pour créer un environnement naturel et dense sans surcharger le code HTML manuellement

Exemple : Les animaux (Gnous) avec l'animation et l'IA

Les gnous sont des éléments vivants de la scène. Leur comportement combine une animation en boucle et un script de déplacement intelligent.

Code HTML : On attache le modèle, l'animation ("walk") et le script d'intelligence artificielle (wildebeest-ai)

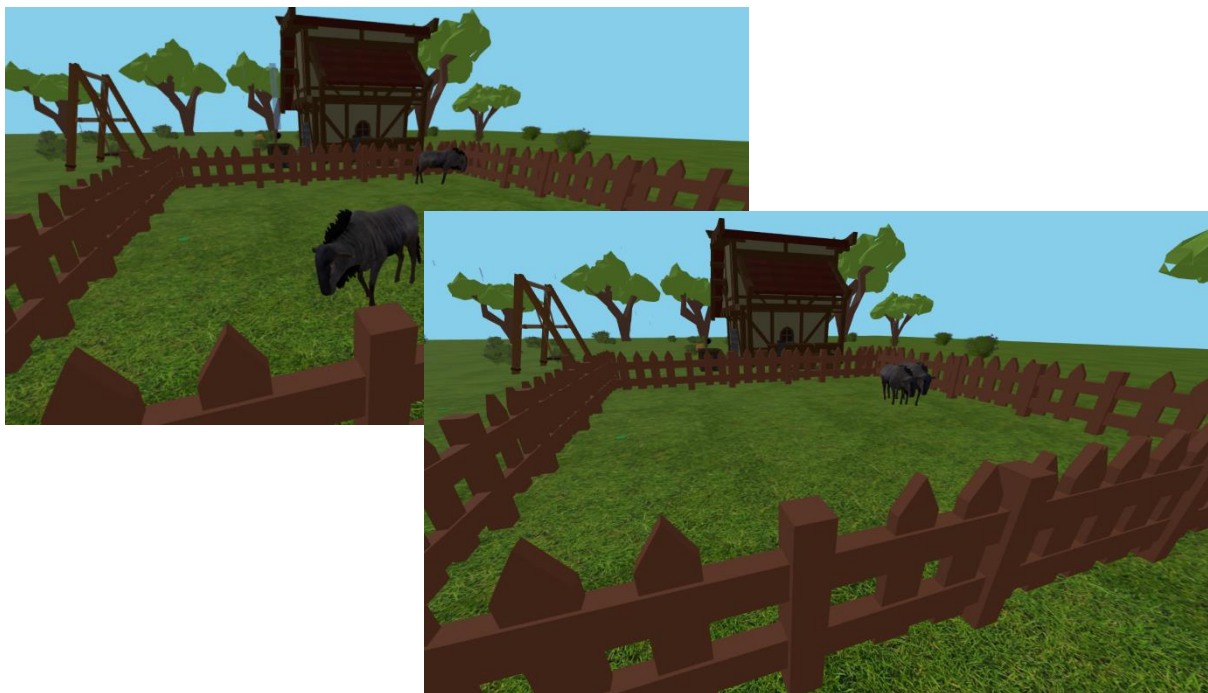
```
<a-entity
  id="gnou1"
  gltf-model="#wildebeestModel"
  position="0 0 4"
  scale="0.7 0.7 0.7"
  animation-mixer="clip: *walk*; loop: repeat; timeScale: 1.0"
  wildebeest-ai
></a-entity>
<a-entity
  id="gnou2"
  gltf-model="#wildebeestModel"
  position="2 0 5"
  scale="0.7 0.7 0.7"
  animation-mixer="clip: *walk*; loop: repeat; timeScale: 0.9"
  wildebeest-ai
></a-entity>
```

Logique de l'IA (wildebeest-ai) : Ce composant gère le déplacement physique. Les gnous avancent tout droit et tournent aléatoirement ou lorsqu'ils atteignent les limites de l'enclos

```
AFRAME.registerComponent("wildebeest-ai", {
  tick: function () {
    // Avance en continu (L'animation de marche tourne en boucle visuellement)
    this.el.object3D.translateZ(-0.02);

    // Gestion des limites (Reste dans l'enclos)
    var pos = this.el.object3D.position;
    if (pos.x < -3.5 || pos.x > 3.5 || pos.z < 0.5 || pos.z > 7.5) {
      // Demi-tour si on touche une barrière virtuelle
      this.el.object3D.rotation.y += Math.PI + (Math.random() - 0.5);
    }
  }
});
```

Donnant alors des animaux vivants en mouvement :



Nous avons aussi également rajouté un chat :

```
<!-- LE CHAT -->
<a-entity
  gltf-model="#chatModel"
  position="-12 0 -11"
  scale="0.0024 0.0024 0.0024"
  rotation="0 90 0"
  animation-mixer="clip: *; loop: repeat; timeScale: 2.0"
  cat-ai
></a-entity>
```

La complexité dans cette fonctionnalité est qu'il est hors de notre champ de compétences de palier non seulement les mouvements des animaux et le système de grab.

En effet il serait évident de pouvoir grab le chat en étant dans la VR mais ayant un mouvement statique fixe cela cause un paradoxe dans l'application qui nécessite beaucoup plus d'investissements et de connaissances dans le domaine

Pour ce qui est de l'environnement, le soleil sol arbres etc... sont directement rajoutés sur la scène en jouant avec leur propriété :

```
<!-- 3. L'ENVIRONNEMENT -->
<!-- Un ciel bleu -->
<a-sky color="#87CEEB"></a-sky>

<!-- Soleil -->
<a-entity
  gltf-model="#sunModel"
  position="-20 50 -30"
  scale="5 5 5"
></a-entity>

<!-- Lumière -->
<a-entity
  light="type: ambient; color: #FFF; intensity: 0.7"
></a-entity>
<a-entity
  light="type: directional; color: #FFF; intensity: 1; castShadow: false"
  position="-1 1 0.5"
></a-entity>

<a-plane
  src="#grassTexture"
  rotation="-90 0 0"
  width="100"
  height="100"
  repeat="20 20"
  material="roughness: 1; metalness: 0"
  class="clickable"
  click-teleport
```

La majorité des objets comme la lumière, la texture du sol et d'autres sont directement gérés par la bibliothèque A-Frame qui est un Framework très puissant et complet

Nous avons également eu recours à l'IA pour y ajouter des propriétés comme pour par exemple empêcher les objets d'être en collision entres elles, d'empêcher les arbres de « spawn » tout proche de la maison etc.

```
function createVegetation() {
  var container = document.querySelector("#vegetation");

  // 1. ARBRES
  var treeCount = 32;
  var trees = [];

  for (var i = 0; i < treeCount; i++) {
    var valid = false,
        attempts = 0;
    var x, z;
    while (!valid && attempts < 50) {
      x = (Math.random() - 0.5) * 80;
      z = (Math.random() - 0.5) * 80;
      var dist = Math.sqrt((x - 0) ** 2 + (z - -10) ** 2);
      if (dist > 25) {
        valid = true;
        for (var t = 0; t < trees.length; t++) {
          if (
            Math.hypot(x - trees[t].x, z - trees[t].z) <
            5
          ) {
            valid = false;
            break;
          }
        }
      }
    }
    attempts++;
  }
}
```

Majeur problème : La physique

En effet, après deux journées de tentative ne nous sommes pas parvenus à gérer la physique de notre monde VR avec quelques « explications » apparentes bien que cela soit bien évidemment possible :

- **L'Effet Tunnel (Tunneling)** : Si un objet tombe trop vite, il peut traverser le sol entre deux images (frames). Le moteur physique ne détecte pas la collision car à l'image T il est au-dessus, et à l'image T+1 il est en dessous.
- **La Complexité des Colliders** : Utiliser la forme réelle d'un objet (Mesh Collider) est trop lourd pour le processeur. On utilise donc des boîtes (Box) ou des sphères approximatives.
Parfois, un objet semble flotter ou entrer en collision avec "du vide" à cause de cette simplification.
- **L'Initialisation (Timing)** : Les objets 3D sont parfois chargés avant que le sol physique ne soit prêt. Donc ils finissent par tomber à travers le monde dès le lancement. Nous avons ajouté un délai (setTimeout) et un état de "sommeil" (sleepy) pour forcer les objets à attendre bien que cela soit inefficace

La physique étant quasi impossible sur navigateur cela nous a pris beaucoup de temps à tester chaque tentative de fix sur la VR.

Communication entre Back-Office et VR :

L'application VR ne se connecte pas directement à la base de données (pour des raisons de sécurité). Mais communique avec le Back-Office via une **API**

Une API (Interface de Programmation d'Application) est un intermédiaire qui permet à deux applications de communiquer. Dans notre projet, elle sert de "serveur" qui livre uniquement les données nécessaires (liste des utilisateurs) au format JSON, sans donner accès à toute la base de données

Dans l'index.html au chargement de la page, un script JavaScript envoie une requête (fetch) vers le serveur du Back-Office :

```
// index.html
fetch("https://selles-francesconi.sio-chopin.fr/Challenge2e/public/api.php")
  .then((response) => response.json())
  .then((data) => {
    // 'data' contient la liste des utilisateurs (JSON)
    users = sanitizeUsers(data);
    createSingleTile();
  });
```

Cette méthode permet de récupérer dynamiquement les utilisateurs créés dans le Back-Office sans modifier le code de la VR.

Ce bloc par exemple envoie le password saisi pour valider l'accès avant d'ouvrir la page VR :

```
document.getElementById('submit-btn').addEventListener('click', () => {
  const password = document.getElementById('password-input').value;
  fetch('https://selles-francesconi.sio-chopin.fr/Challenge2e/public/login.php', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ username: selectedUser.username, password: password })
  })
  .then(response => response.json())
  .then(data => {
    if (data.success) {
      window.location.href = 'https://selles-francesconi.sio-chopin.fr/Challenge2eVR/vr.html';
    } else {
      alert('Incorrect password');
    }
  })
  .catch(error => {
    console.error('Login error:', error);
    alert('Login service unavailable');
  });
});
```

On dispose également d'une requête API depuis le back-office pour déterminer quels utilisateurs peuvent accéder à la VR et ainsi envoyer à la VR les bons identifiants de connexions s'il faut renvoyer une erreur en cas de credentials incorrects :

```
// API Login for VR app
public function apiLogin()
{
    header('Content-Type: application/json');
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        $username = $_POST['username'] ?? '';
        $password = $_POST['password'] ?? '';

        $user = $this->userModel->findByUsername($username);
        if ($user && password_verify($password, $user['password'])) {
            if ($user['idWorld'] == 1) { // Only allow if world is La prairie pluvieuse
                $avatar = $this->avatarModel->findById($user['idAvatar']);
                // Map model names to VR assets
                $modelMap = [
                    'adventurer.glb' => 'aventurier.glb',
                    'astronaut.glb' => 'astronaute.glb',
                    'fitness.glb' => 'fitness.glb',
                    'mako.glb' => 'shark.glb',
                    'pug.glb' => 'pug.glb'
                ];
                $vrModel = $modelMap[$avatar['modelAvatar']] ?? $avatar['modelAvatar'];
                echo json_encode([
                    'success' => true,
                    'avatar' => [
                        'model' => $vrModel,
                        'name' => $avatar['nameAvatar']
                    ]
                ]);
            } else {
                echo json_encode(['success' => false, 'message' => 'Access denied: Only users with world "La prairie pluvieuse" can access VR.']);
            }
        } else {
            echo json_encode(['success' => false, 'message' => 'Invalid credentials.']);
        }
    } else {
        echo json_encode(['success' => false, 'message' => 'Method not allowed.']);
    }
    exit();
}

// API Get non-admin usernames
public function apiGetUsernames()
{
    header('Content-Type: application/json');
    $users = $this->userModel->findAll();
    $usernames = [];
    foreach ($users as $user) {
        if ($user['userRole'] !== 'ADMIN') {
            $usernames[] = $user['username'];
        }
    }
    echo json_encode($usernames);
    exit();
}

// Logout
public function logout()
{
    session_destroy();
    header('Location: index.php?page=home');
    exit();
}
}
```

Problèmes rencontrés :

- Physique dans la VR non fonctionnelle (expliquée plus haut)
- Emplacements et propriétés des objets compliqués à manipuler sur PC, en effet nous avons découverts trop tard qu'il existait des outils pour manipuler directement les objets de notre monde VR depuis le navigateur, nous avons modifié pour la grande majorité du projet les propriétés des objets via code et refresh de la page, causant une perte de temps considérable
- Difficultés avec le système de grab (physique/hitbox complexe/paradoxe mouvement des entités)
- Téléportation fonctionnelle sur le site mais non sur la VR, la différence de fonctionnement entre les deux systèmes a causé beaucoup de confusions
- Ayant tous travaillé sur le même monde les merges ont causé beaucoup de conflits et donc de temps, l'organisation est à retravailler
- Impossibilité de cast en effet notre manette gauche a eu quelques difficultés technique nous empêchant alors de tester parfois la téléportation
- Manque de connaissances, la deadline instaurée et la pression de résultat a eu une production à effet inverse sur l'ensemble du groupe par moment

Conclusion

Conclusion du projet :

Le back-office est fonctionnel et n'a causé aucun problème lors de sa production, la VR quant à elle fonctionne bien qu'elle ne réponde pas à toutes les attentes du cahier des charges, l'objectif de relier les 5 mondes n'a pas abouti également. Néanmoins la présence de plusieurs entités et des animaux qui bougent donnent quand même le ressenti d'être dans un monde naturel et amusant au premier abord pour l'utilisateur même si beaucoup d'améliorations sont à faire.

Conclusion générale du challenge :

Bien que beaucoup de fonctionnalités ne sont pas opérationnels et souvent pour des raisons que l'on ignore (téléportation, physique...)

L'activité était pour l'ensemble de l'équipe très intéressante et instructive et l'introduction à la réalité virtuelle nous ont permis de comprendre que ce n'était pas si inatteignable que ça étant donné qu'auparavant la réalité virtuelle était pour nous quelque chose de très complexe à mettre en place même si naturellement les mondes développés requièrent énormément de compétences dans le domaine

Le projet nous a permis pour chacun de mettre en action nos compétences acquises au cours de notre cursus du BTS et de voir comment nous réagissons à la pression d'un domaine qui nous échappe ainsi que d'un travail de groupe non habituel (Merges Github) qui dans la finalité ne peut être que bénéfique dans le cadre de notre progression dans l'informatique.

Kamdine HASSANZADEH

Quentin JEANDEL

Sael SELLES